SUMMATIVE PROJECT

JavaScript Programmer

SkillsTrack Learner Support Portal

A two-month JavaScript, Firebase and GitHub development project

Project Information	Details
Skills Programme	JavaScript Programmer
Skills Programme ID	SP-220373
Curriculum Code	900219-000-00-00
NQF Level / Credits	NQF Level 4 / 60 Credits
Project Duration	2 Months
Assessment	Month 1: 192 marks | Month 2: 190 marks
Total	382 marks
Database	Firebase Realtime Database
Project Method	Team development with individual evidence of competence

Competence Threshold and Academic Integrity
The competence threshold for this assessment is 75%. However, achieving 75% does not automatically mean that a learner will be declared competent.
The assessor must also be satisfied that the learner understands the work submitted and can explain how the solution was planned, developed and applied. A learner may obtain 75% or more but still be declared Not Yet Competent if they cannot explain their code, technical decisions, development process or individual contribution.
Learners must not:

- **copy and paste code or answers without understanding them;**
- **allow one group member to complete all the work;**
- **submit another person's work as their own; or**
- **use ChatGPT, Google Gemini or any other artificial intelligence tool to complete the project without understanding and being able to explain the output.**

All group members will be assessed and marked individually. Each learner must be able to explain their own contribution and demonstrate an understanding of the full project.
The assessor will ask oral questions before, during or after the assessment to evaluate the learner's understanding, problem-solving ability and application of knowledge.
Any confirmed cheating, plagiarism, unauthorised collaboration or submission of work that is not the learner's own will result in a mark of zero.

Learner name: Lwandle, Hilda and Onkarabetse
Team name: Squad 2
Assessor: Londeka Zikalala
Project start date: 03/08/2026 Final due date: 03/09/2026

Month	Marks
Month 1	/192
Month 2	/190
Total	/382

1. Project Purpose and Programme Alignment

The purpose of this project is to provide learners with an integrated opportunity to solve a realistic operational problem using JavaScript and applicable development tools. The finished product must be a functional, readable and testable front-end application supported by Firebase data, REST communication and disciplined version control.

The project addresses the programme requirement to create well-written JavaScript solutions, test and debug code, communicate with web servers, and work collaboratively through Git and GitHub. **It combines the knowledge and practical skills developed during the Introduction, Principles, Intermediate Programming and Projects with JavaScript modules.**

1.1 Project Assessment Principles

- The project is assessed in two monthly submissions.
- Each learner must submit individual evidence even when the application is developed in a team.
- A feature may receive no marks if the learner cannot explain, demonstrate or modify the submitted code.
- The final mark is calculated out of 382 marks
- All source code, project documentation and Git history must be available to the assessor.

2. Project Narrative and Client Brief

SkillsTrack Training Centre supports learners who attend short occupational programmes. Learners currently record goals, tasks, support bookings and progress in separate documents and messages. Assessors struggle to see what work is outstanding, which learners need support, and whether learners are making progress.

The centre requires a browser-based Learner Support Portal. The portal must allow registered users to manage learning tasks, book support sessions, review progress, use learning resources and complete a short coding game. The system must use JavaScript for application logic and Firebase as the database.

2.1 Client Objectives

- Provide a single, clear interface for managing learning tasks and support requests.
- Allow users to register, sign in and view their own information.
- Store, retrieve, update and delete application data in Firebase.
- Provide meaningful calculations and summaries based on stored data.
- Include an interactive mini-game that reinforces basic programming concepts.
- Allow a team of developers to collaborate through GitHub and controlled version history.

2.2 Required Users

User	Required access
Learner	Register and sign in; manage own tasks; book support; view progress; play the mini-game; print a progress summary.
Assessor or administrator	View submitted support bookings and relevant learner activity; update booking status where included in the approved scope.

2.3 Minimum Application Features

- User registration, sign-in, sign-out and authenticated user state.
- Dashboard displaying task totals, completed work, outstanding work and calculated progress.
- Task manager with create, read, update and delete functions.
- Support-session booking form with validation and status feedback.
- Search, filter or sort functionality using arrays and higher-order functions.
- Cookie-based preference such as theme, display mode or last selected filter. Passwords and sensitive data may not be stored in cookies.
- Confirmation dialog before a destructive action, a redirect after an appropriate action, and a printable progress summary.
- At least one animation driven by JavaScript timers and one controlled multimedia element.
- A basic operable mini-game created with a Assessor-approved JavaScript framework or library.

- Firebase Realtime Database integration and documented REST API CRUD operations.

3. Required Tools, Technologies and Evidence

Area	Required technology or evidence
Core development	HTML5, CSS3 and JavaScript (ES6 or later).
IDE	Visual Studio Code or another Assessor-approved IDE with formatting, linting and debugging support.
Database	Firebase Realtime Database.
Authentication	Firebase Authentication or another Assessor-approved Firebase authentication method.
REST communication	Firebase Realtime Database REST API using GET, POST, PUT or PATCH, and DELETE. Requests must be authenticated or use a Assessor-controlled test environment.
Framework or library	One Assessor-approved JavaScript framework or library must be used. The game may use Phaser.js, Kaboom.js or another approved option.
Version control	Git and GitHub, including repository, branches, commits, pull requests, merges and contribution history.
Continuous integration	A basic GitHub Actions workflow or equivalent automated check, such as linting or tests.
Testing	Browser developer tools, breakpoints, console, stack traces, manual test cases and evidence of corrections.

3.1 Firebase Data Requirements

The team must design a Firebase data structure before coding. A suggested structure is shown below; teams may adapt it with Assessor approval.

Path	Purpose	Example fields
users/{uid}	Basic learner profile and application role.	displayName, email, role, createdAt
tasks/{taskId}	Learning tasks owned by a user.	userId, title, category, dueDate, priority, completed, createdAt
bookings/{bookingId}	Support-session requests.	userId, topic, preferredDate, notes, status
scores/{scoreId}	Mini-game or quiz results.	userId, score, duration, completedAt
resources/{resourceId}	Optional learning resources.	title, type, url, description

3.2 Security and Data Rules

- Database rules must not permit unrestricted public write access.
- Users should access only data appropriate to their authenticated identity and role.
- Passwords must be handled by Firebase Authentication and may not be stored in the database, cookies or source code.
- Secrets, private credentials and service-account files may not be committed to GitHub.
- User input must be validated before it is written to Firebase.

4. Team Collaboration and Individual Accountability

The project must be completed in teams of three learners unless the Assessor approves another arrangement. Collaboration is assessed, but competence is decided individually.

- Create one shared GitHub repository and a project board or issue list.
- Assign features through issues or tasks and identify the responsible learner.
- Use feature branches rather than making all changes directly on the main branch.

- Open pull requests, conduct at least one peer review per learner, and merge approved work.
- Each learner must make meaningful commits and be able to explain their contribution.
- The team must resolve at least one merge or integration issue and record how it was handled.

4.1 Assessor Review Sessions

- A review must be booked on the provided roster.
- Review appointments are allocated on a first-come, first-served basis.
- You project must be reviewed by the assessor as per the monthly due dates
- The team must present a working version, Git history and specific questions.
- Feedback must be recorded, considered and reflected in the following submission.

5. Month 1 - Planning, Tools and Core JavaScript (192 marks)

During Month 1, the team must analyse the problem, plan the solution, configure the development environment, establish the GitHub workflow and create the functional application foundation. The submission must demonstrate understanding of JavaScript fundamentals as well as practical use of variables, functions, arrays, loops, operators, conditionals, DOM events and debugging tools.

5.1 Month 1 Work Schedule

Week	Focus	Expected progress
1	Problem analysis and programming life cycle	Requirements, user stories, flowcharts, pseudocode, scope and testable acceptance criteria.
2	Environment, Git and architecture	IDE configuration, repository, branches, CI check, Firebase data model and class design.
3	Core JavaScript implementation	Variables, functions, arrays, loops, calculations, conditionals, events and dynamic DOM output.
4	Application foundation and review	Interface shell, validation prototype, preferences, debugging evidence, Assessor review and Month 1 submission.

5.2 Month 1 Deliverables

- Problem statement, project scope and client requirements.
- At least six user stories with acceptance criteria.
- Programming life-cycle plan showing analysis, design, coding, testing, implementation review and improvement.
- Flowcharts or pseudocode for login, task creation, progress calculation and deletion confirmation.
- Firebase data model and REST endpoint plan.
- Class or object design showing at least two planned classes and their relationships.
- Configured IDE with formatter, linter and debugging tools.
- Shared GitHub repository with README, .gitignore, issues, branches, pull request evidence and basic CI check.
- Working application shell with navigation and dynamic DOM content.
- JavaScript evidence using variables, data types, operators, functions, scope, arrays, loops and conditionals.
- Cookie preference, dialog or confirmation, redirect and print-function plan or prototype.
- Testing and debugging record with at least three issues and corrections.
- Assessor review record and Month 1 reflection.

5.3 Month 1 Coding Requirements

- Use strict mode in JavaScript modules or scripts where appropriate.
- Use let and const correctly and avoid unnecessary global variables.
- Create reusable functions with parameters and return values, including at least one arrow function.
- Use arrays of data objects and demonstrate map, filter or reduce for a meaningful application result.
- Use loops and conditional statements to process and display data.

- Use event listeners rather than relying only on inline event attributes.
- Create, update or remove at least one DOM element dynamically.
- Apply clear naming, indentation, comments and file organisation.

5.4 Month 1 Formative Questions (92 Marks)

1. Explain why the programming life cycle should be followed before coding begins. (2 marks)

The programming life cycle should be followed before coding begins to ensure the problem is clearly understood and well-prepared, allowing us to use our time effectively which in return makes the final program easier to test and debug ensuring that the finished program satisfies users requirements.

2. List and explain the main steps of the programming life cycle. Your answer must show how each step contributes to the development of a working solution. (15 marks)

1) Problem Analysis / Requirements: Understand what the user needs, including inputs, processing . and outputs.
Contribution: Ensures the correct problem is being solved.

2) Design: Plan the solution using algorithms, pseudocode, flowcharts and diagrams.
Contribution: Provides a clear plan and helps find logic errors before coding.

3) Coding / Implementation: Convert the design into actual program code.
Contribution: Creates the working software.

4) Testing: Run test cases to check that the program works correctly.
Contribution: Finds bugs and checks that requirements are met.

5) Debugging: Find and fix errors discovered during testing.
Contribution: Makes the program more reliable and accurate.

6) Documentation: Record information about the code and how to use the program.
Contribution: Helps developers maintain the program and users operate it correctly.

7) Deployment: Release the completed program for users.
Contribution: Makes the solution available for real-world use.

8) Maintenance: Fix bugs and update the program after release.
Contribution: Keeps the software useful, secure and up to date.

3. Explain when *const* should be used instead of *let*. Also explain why *var* should normally be avoided in modern JavaScript. (5 marks)

We use const when a variable's value doesn't need to be reassigned after its first declared, meaning the value assigned will remain fixed throughout the program. On the other hand, let is used when the value stored in the variable is expected to change while the program runs. Var should be avoided in modern JavaScript because it has some weird and confusing behavers that can cause bugs such as ignoring block scope, allowing redeclaration and hoisting can be confusing.

4. Explain how local and global scope can affect the reliability and maintainability of a JavaScript application. **(2 marks)**

A variable declared with local scope can only be accessed inside the function in which declared so other parts of the program cannot change it this makes the application more reliable and predictable.A variable declared with local scope can be accessed and changed from anywhere in the program increasing the risk of conflicts since any function could unexpectedly alter its value.

5. Explain how map(), filter() and reduce() process an array of task objects differently. Provide one suitable use for each method. **(6 marks)**

map(): Transforms every element in the array and returns a new array of the same length, but each item has been changed in some way. Suitable use: getting all task IDs task.map(task => task.id).

filter(): checks a conduction and returns only the elements that pass it. Suitable use: get completed tasks tasks.filter(task => task.completed).

reduce(): processes all elements and combine them into accumulated value. Suitable use: find the total cost of all tasks tasks.reduce((Total), Task) => total task.cost, 0)

6. Explain why an application should use classes or structured objects instead of storing related information in several unrelated variables. **(5 marks)**

Group related data together, making code easier to read.
Improve maintainability because related information is kept in one place.
Allow code reuse by creating multiple objects from the same class.
Reduce errors by keeping related properties together.
Make programs easier to scale because new objects can be created without repeating variables.

7. Explain how branches, pull requests and automated checks reduce risk when developers collaborate on a project. **(5 marks)**

Branches allow developers to work on a feature or fix any errors/bugs without affecting the main code.
Pull request allow team members to review changes and identify mistakes before merging.
Automated checks run tests, linters and builds to detect errors automatically.
Improve code quality ensure changes follow coding standards and work correctly.
Reduce risk changes are isolated reviewed and tested before reaching the main branch.

8. Study the following values:

```
const userName = "Lerato";
const age = 22;
const isActive = true;
const selectedProject = null;
```

Answer the following:

a. State the data type of each value. **(4 marks)**

String
Number
Boolnean
Object

b. Explain how age could be converted from a number to a string. **(1 mark)**

Age can be converted to string using String(age)
const age = String (22);

c. Explain what the typeof operator is used for. **(1 mark)**

The typeof operator is used to check and return the data type of a value or variable

9. Analyse the following code:

```
const total = "10" + 5;
const looseComparison = 5 == "5";
const strictComparison = 5 === "5";

console.log(total);
console.log(looseComparison);
console.log(strictComparison);
```

Answer the following:

a. State the output of each console.log() statement. **(3 marks)**

105
True
False

b. Explain why "10" + 5 does not produce the number 15. **(2 marks)**

When the + operator is used in JavaScript and at least one operand is a string, the operation results in string concatenation instead of numerical addition. In this case, the number 5 is converted to the string "5" and appended to the end of "10", creating the string "105" rather than performing a numeric sum.

c. Explain the difference between == and ===. **(2 marks)**

Compare two values after automatically converting them to the same data type if necessary..
Compares both the value and the data type without performing any type of conversion.

d. Rewrite the first statement so that it produces the number 15. **(1 mark)**

const total = 10 + 5;

10. Study the following functions:

```
function calculateTotal(price, quantity = 1) {
        return price * quantity;
}

const applyDiscount = (total, percentage) => {
        return total - total * (percentage / 100);
};

const orderTotal = calculateTotal(150, 3);
const finalTotal = applyDiscount(orderTotal, 10);
```

Answer the following:

a. Identify the parameters of calculateTotal(). **(2 marks)**

The parameters of calculateTotal() are price and quantity

b. Explain the purpose of the default value assigned to quantity. **(1 mark)**

If calculateTotal() is called without a value for quantity, it automatically defaults to 1, this ensures that function still works correct and prevents errors such as undefined when the user input for quantity is not provided

c. Explain what the return keyword does. **(1 mark)**

Return sends a value back to the place where the function was called and immediately stops the function from executing any futher code.

d. State the value stored in orderTotal. **(1 mark)**

450

e. State the value stored in finalTotal. **(1 mark)**

405

f. Explain one difference between a function declaration and an arrow function. **(2 marks)**

Function declaration is hoisted, meaning it can be called earlier in the code than the line in which it is defined. An arrow function is not hoisted in the same way meaning it cannot be called before its definition line runs.

11. Study the following array:

```
const tasks = [
    { title: "Create wireframes", completed: true, hours: 3 },
    { title: "Develop login form", completed: false, hours: 5 },
    { title: "Test application", completed: true, hours: 2 },
    { title: "Write documentation", completed: false, hours: 4 }
];
```

Write JavaScript code that:

a. Uses a loop to display the title of every task. **(2 marks)**

```
for (const task of tasks) {
    console.log(task.title);
}
```

b. Uses a conditional statement to display only completed tasks. **(2 marks)**

```
for (const task of tasks) {
  if (task.completed) {
    console.log(task.title);
}
```

c. Counts the number of completed tasks. **(2 marks)**

```
const countCompleted = tasks.filter(task => task.completed).length;
console.log(countCompleted;
```

d. Calculates the total number of hours for all tasks. **(2 marks)**

```
const hoursTotal = tasks.reduce((sum, task) => sum + task.hours, 0);
console.log(hoursTotal);
```

e. Displays an appropriate message if no tasks are available. **(2 marks)**

```
if (tasks,length === 0)  {
  console.log("No tasks are available.");
}
else{
}
```

15. Study the following statement:

```
document.cookie = "theme=dark; max-age=3600; path=/";
```

Answer the following:

a. Explain what information the cookie stores. **(1 mark)**

It stores a cookie named theme with the value dark a small piece of key-value data saved in the user's browser so their preferred them colour can be called on future visits.

b. Explain the purpose of max-age=3600. **(1 mark)**

To set how long in seconds the cookie will be remembered before it expires and is deleted by the browser 3600 seconds after it was created.

c. Explain the purpose of path=/. **(1 mark)**

The purpose of path=/ is the make the cookie assessable from every page across the entire website, rather than being restricted to one specific folder or page.

d. Write a JavaScript statement that displays the available cookies. **(1 mark)**

```
console.log(document.cookie);
```

Testing and Problem-Solving

16. A registration form contains the following fields:

- name;
- email address;
- password; and
- age.

Develop five test cases that could be used to test the form. Each test case must include:

- the input or condition being tested; and
- the expected result.

Your test cases must include at least one valid submission, one missing value, one invalid email address and one boundary-value test.

(15 marks)

1) Test Case:	Valid submission:
Input/condition Tested:	Name: "Thabo Nkosi", Email: "thabo@example.com" Password: "Secure@123", Age: 25 (all fields correctly filled in)
Expected Result:	Form is accepted and submitted successfully.

2) Test Case:	Missing value
Input/Condition Tested:	Name field is left blank; Email, Password and Age are all filled in correctly
Expected Result:	Form is rejected and “Name is required” is displayed.
3) Test Case:	Invalid email
Input/Condition Tested:	Email entered as "thabo.example.com" (missing the @ symbol / not a valid email format); other fields valid
Expected Result:	Form is rejected and “Please enter a valid email address” is displayed.

4) Test Case:	Boundary value
Input/Condition Tested:	Age entered as 17, one year below the minimum allowed age of 18; other fields valid
Expected Result:	Form is rejected because the age is below the minimum. Age 18 should be accepted.

5) Test Case:	Invalid password
Input/Condition Tested:	Password entered as "123" (shorter than the required minimum length, e.g. under 8 characters); other fields valid
Expected Result:	Form is rejected and a password-length error is displayed.

6. Month 1 Assessment Rubric

Criterion	Assessment standard	Marks
Problem analysis and requirements	Problem, users, scope, constraints, user stories and acceptance criteria are complete and appropriate.	/6
Programming life-cycle plan	Development stages, milestones, iteration and testing activities are planned logically.	/6
Firebase data and REST planning	Data paths, fields, ownership and planned CRUD operations are accurately documented.	/8
Object-oriented design	Classes or structured objects, properties, methods and relationships are planned and justified.	/8
IDE and toolkit configuration	IDE, extensions, formatter, linter, debugger and selected framework or library are configured and demonstrated.	/6
GitHub repository and collaboration setup	Repository, README, .gitignore, issues, branch workflow, pull request and basic CI check are established.	/10
Syntax and disciplined coding style	Code is concise, consistently formatted, meaningfully named, commented and organised.	/6
Variables, data types and operators	Variables, scope, types, conversions and operators are applied correctly.	/8
Functions and scope	Reusable functions, parameters, returns, arrow functions and scope are applied correctly.	/10
Arrays, loops and conditionals	Arrays of objects, array methods, loops and decision structures solve project-relevant problems.	/12
DOM and event foundation	Events and DOM operations produce clear, dynamic and correct application behaviour.	/8
Cookies, dialogs, redirect and print	Safe preference storage and appropriate browser functions are planned or demonstrated.	/4
Testing and debugging evidence	At least three issues are diagnosed with suitable tools, corrected and retested.	/4
Review, reflection and explanation	Feedback is recorded and the learner explains the Month 1 decisions and contribution.	/4
Month 1 summative knowledge question	Summative knowledge question answered and correct.	/92
	Total:	/192

Comments:_____

7. Month 2 - Intermediate Development, Firebase and Collaboration (190 marks)

During Month 2, the team must complete the application and integrate authentication, Firebase data, REST requests, object-oriented programming, validation, error handling, debugging, animation, multimedia and a basic JavaScript game. GitHub must show genuine team collaboration and controlled version history.

7.1 Month 2 Work Schedule

Week	Focus	Expected progress
5	Authentication and Firebase	Firebase project, secure rules, authentication, user state and initial database records.
6	CRUD, REST and DOM	Create, read, update and delete operations; dynamic rendering; search, filter and calculated summaries.
7	Intermediate features	OOP implementation, validation, error handling, animation, multimedia and basic game.
8	Integration and assessment	Testing, refactoring, pull requests, final integration, technical review and Month 2 submission.

7.2 Month 2 Deliverables

- Completed and operable Learner Support Portal that meets the approved brief.
- Firebase Authentication with registration, sign-in, sign-out and authenticated user state.
- Firebase Realtime Database with secure rules and structured data.
- Full CRUD for at least one primary entity, such as learning tasks, through Firebase REST requests.
- Evidence of POST, GET, PUT or PATCH, DELETE and a final GET or equivalent verification.
- Object-oriented implementation using ES6 classes, object instances and at least one relationship such as inheritance or composition.
- Client-side validation for names, passwords, email, numeric fields and required data.
- Error handling using try, catch, finally and at least one custom thrown error.
- Debugging and refactoring evidence showing before-and-after code.
- Dynamic DOM operations that create, change and remove elements or attributes.
- JavaScript timer-based animation and controlled image, audio or video content.
- An operable basic game using a Assessor-approved JavaScript framework or library.
- GitHub evidence showing individual branches, commits, pull requests, reviews and merges.
- Final test report, README, contribution report, Assessor review and learner reflection.

7.3 Required Functional Behaviour

Feature	Minimum acceptable behaviour
Authentication	Valid users can register, sign in and sign out. Invalid input produces clear feedback. Authenticated state controls access to personal features.
Task management	Users can create, view, edit, complete and delete tasks. Deletion requires confirmation.
Progress calculation	The application calculates completed, outstanding and overdue work from task data and displays the result dynamically.
Support booking	Users can submit a validated support request and view an appropriate success or error response.
Search and filters	Users can search, filter or sort tasks or resources using array methods and reusable functions.
Preferences	A non-sensitive preference is written to, read from, modified in and removable from a cookie.

Feature	Minimum acceptable behaviour
Print and redirect	Users can print a suitable summary and are redirected only when the action and application flow justify it.
Game	The mini-game is playable, records a score or outcome and stores the result or summary in Firebase.

7.4 Month 2 Formative Questions (90 Marks)

1. Explain how the Firebase data model can prevent one user's records from being confused with or displayed as another user's records. Refer to user IDs and document ownership in your answer.(6 marks)

2. Explain the difference between client-side validation and authentication. Provide one example of each from the application. (6 marks)

3. Explain when POST, GET, PUT or PATCH, and DELETE should be used in a REST workflow. Provide one application-related example for each method. (8 marks)

4. Explain how try, catch, finally, and throw work together when a Firebase or REST request fails.(8 marks)

5. A learner's application displays a progress total of 150% instead of 75%. Describe the steps that should be followed to identify and correct the logical error. Refer to debugging tools and the flow of data in your answer. (8 marks)

6. Explain how pull requests and peer reviews can prove individual contribution and improve the quality of a team's code. (6 marks)

7. Firebase Security and Access Control

A Firestore database stores learner tasks using the following structure:

```
users
  └── userId
      └── tasks
          └── taskId
```

a. Explain why the authenticated user's ID should be used in the document path. **(2 marks)**

b. Explain how Firebase Security Rules can restrict access to the correct user. **(3 marks)**

c. State one risk of allowing unrestricted read and write access to the database. **(1 marks)**

d. Explain why validation must still be performed even when security rules are present. **(2 marks)**

_____
_____
_____

8. Analyse the following JavaScript error-handling code:

```
async function saveTask(task) {
  try {
    if (!task.title) {
      throw new Error("A task title is required.");
    }

    const response = await fetch("/tasks", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(task)
    });

    const result = await response.json();
    return result;
  } catch (error) {
    console.log("Task saved successfully.");
  } finally {
    console.log("Request completed.");
  }
}
```

a. Explain the purpose of the async keyword. **(1 marks)**

_____
_____

b. Explain the purpose of await. **(1 marks)**

_____
_____

c. Explain why a custom error is thrown when the task has no title. **(2 marks)**

_____
_____
_____
_____

d. Identify the incorrect message in the catch block and explain why it is misleading. **(2 marks)**

_____
_____
_____
_____

e. Rewrite the catch block so that it reports the actual error. **(2 marks)**

_____
_____
_____

f. Explain when the finally block will execute. **(2 marks)**

9. DOM Manipulation **(8 marks)**

Study the following HTML:

```
<ul id="taskList"></ul>
<button id="addTaskButton">Add Task</button>
```

Explain how JavaScript could be used to create a new <li> element, add task text to it, append it to the list, and later remove it. Name the relevant DOM methods or properties.

10. Object-Oriented JavaScript

Study the following class:

```
class Task {
 constructor(title, dueDate) {
  this.title = title;
  this.dueDate = dueDate;
  this.completed = false;
 }

 markComplete() {
  this.completed = true;
 }
}
```

a. Explain the purpose of the Task class. **(2 marks)**

b. Explain what the constructor does. **(2 marks)**

c. State the three properties created for every task object. **(2 marks)**

d. Explain what happens when markComplete() is called. **(1 marks)**

e. Write one statement that creates a new Task object. **(1 marks)**

11. Explain how JavaScript timers such as setTimeout() and setInterval() differ. Describe one suitable use for each in a web application, and state one consideration when adding animation or multimedia.(6 marks)

12. A learner submits a form that creates a new record in Firebase. The application must then retrieve the record and display it on the page. Describe the complete asynchronous workflow from the moment the user selects the submit button until the new record appears in the interface. Refer to validation, the database request, the response, DOM updates, loading feedback, and error handling. (8 marks)

8. Month 2 Assessment Rubric

Criterion	Assessment standard	Marks
Functional solution	The completed application solves the stated problem and all major user journeys operate correctly.	/6
Object-oriented implementation	Classes, objects, properties, methods and relationships are implemented appropriately and explained.	/8
Firebase database and CRUD	Data is structured, secure and correctly created, retrieved, updated and deleted in Firebase.	/14
Authentication and validation	Registration, login, authenticated state and input validation operate correctly with useful feedback.	/8
REST API communication	Documented GET, POST, PUT or PATCH and DELETE requests communicate correctly with Firebase or an approved web service.	/10
DOM and event handling	The application dynamically creates, changes and removes interface content through clear event-driven code.	/8
Error handling, debugging and refactoring	Errors are handled using appropriate mechanisms; bugs are diagnosed; code is improved without changing intended behaviour.	/8
Animation and multimedia	JavaScript timers or animation functions and controlled multimedia are purposeful and functional.	/6
Basic framework-based game	An operable JavaScript game uses an approved framework or library and records a meaningful outcome.	/8
Data processing and calculations	Functions, arrays, map/filter/reduce, loops and operators produce accurate project-relevant results.	/6
Code quality, tests and documentation	Code is readable, modular and tested; README and test evidence allow the solution to be reviewed and run.	/6
GitHub teamwork and version control	Each learner demonstrates branches, commits, pull requests, reviews, merges and collaborative problem solving.	/10
Technical demonstration and reflection	The learner demonstrates the solution, answers questions and reflects on contribution and improvements.	/2
Month 2 summative knowledge question	summative knowledge question answered and correct.	/90
	Total:	/190

Comments:_____
_____
_____
_____
_____
_____
_____
_____
_____
_____

13. Academic Integrity and Authenticity Declaration

I declare that:

- the evidence submitted for assessment accurately represents my **own contribution;**
- I have acknowledged tutorials, libraries, code examples, artificial-intelligence assistance and other external sources;
- I understand the JavaScript, Firebase and GitHub work that I am presenting;
- I can explain, test and modify the code attributed to me;
- I have not stored passwords, private credentials or confidential data in the repository;
- I understand that if I cannot demonstrate understanding OR provide enough evidence, my project could not be accepted and therefor I will not pass the FISA (Final Integrated Supervised Assessment)

Learner name: _____

Learner signature: _____

Date: _____

Assessor name: _____

Assessor signature: _____

Date: _____

13.1 Source Alignment

This project was developed with reference to the QCTO JavaScript Programmer Skills Programme Document (SP-220373) and the JavaScript Programmer Curriculum Document (Curriculum Code 900219-000-00-00). The detailed monthly rubrics contextualise the programme outcomes and practical skills for an integrated two-month application project.